

mindXtrain — production blueprint for the AMD × lablab.ai hackathon (May 4-10 2026)

The clock is already running. The lablab.ai AMD Developer Hackathon opened its online build window on May 4 2026, the on-site finale runs May 9-10 in San Francisco at the MindsDB SF AI Collective, and the prize pool is **\$21,500+ plus an AMD Radeon AI PRO R9700 GPU** across three tracks (Agents, Fine-Tuning on AMD GPUs, Vision/Multimodal) with stackable cross-cutting prizes for **Best Use of Qwen** and **Build in Public**, and a Hugging Face Spaces "most likes" prize topped by a Reachy Mini Wireless robot. mindXtrain enters as the **Fine-Tuning track** primary, cross-submitted to Best-Use-of-Qwen and Build-in-Public, with a one-line pitch nobody else in the live `lablab-ai-amd-developer-hackathon` HF org currently occupies: *"the first one-command Qwen3.6 fine-tuner natively optimized for MI300X — auto-selects Composable Kernel, AITER, hipBLASLt and Flash-Attention-ROCM configs from a 60-second micro-benchmark, trains via Optimum-AMD + TRL, quantizes with AMD Quark, and serves on vLLM-ROCM 7.2.1, all from a single CLI."* That positioning hits all four explicit lablab judging criteria — Technology Integration, Presentation, Business Value, Originality — and it directly maps onto AMD's actual KPI of ROCm developer adoption, which matters because the judge bench is led by **Ramine Rozen, CVP of AI at AMD**. The remainder of this document is the production blueprint.

LabLab + 4

How the hackathon actually scores, and the angle that wins it

The lablab page lists four criteria with no explicit weights; sister AMD events (Bengaluru, Delhi) reveal a strong implicit theme of **production-readiness under hardware constraints** — judges reward submissions that exploit MI300X's 192 GB HBM3 to do something an H100 80 GB cannot. The current bar to beat is **REPOMIND**, a repo-scale coding agent in the live HF org built on Qwen3-Coder-Next-FP8 + vLLM ROCm 7 that explicitly markets "H100 OOMs on this workload, MI300X just runs it." mindXtrain's differentiator is orthogonal — it is *infrastructure that produces specialized Qwen3 models cheaply on MI300X*, not a single specialized application — and the AutoML/AutoTrain niche is genuinely empty in the lablab archive. The submission therefore needs to be **demonstrable in 90 seconds**, must produce a tangible artifact (a finetuned Qwen3 checkpoint pushed to the HF org, served on a public Space), and must show the AMD stack being *fully* exercised at every layer rather than treated as a black box. The Build-in-Public extension costs almost nothing — two technical X posts tagging @lablab and @AIatAMD plus an MIT-licensed repo are required anyway — so it is a free additional prize pool. `AMD + 7`

The **single most differentiating angle** is the auto-selection layer. No competitor framework — not Axolotl, LLaMA-Factory, Unsloth, torchtune, Primus, or Optimum-AMD itself — runs a **per-job MI300X micro-benchmark** before training to pick CK vs Triton attention backends, hipBLASLt heuristic vs rocBLAS path, AITER vs reference MoE kernels, NCCL_MIN_NCHANNELS, gradient-checkpointing strategy, FSDP shard width, and LoRA rank

against the actual (model, dataset shape, sequence length, GPU count) tuple. mindXtrain owns that AOT-only autotune layer.

The mindXtrain reference stack, pinned

The recommended container is `rocm/primus:v26.2` (which is identical to `rocm/pytorch-training:v26.2`) and bundles ROCm 7.2.1, PyTorch 2.9.1, AOTriton 0.11+, AITER 0.1.12, RCCL with Pollara fixes, Apex with fused RoPE, Megatron-Core, TorchTitan, Primus-Turbo, ROCm/TransformerEngine, and Quark). Pull it once, snapshot the SHA256 digest into `infra/podman/digest.lock`, and never run training off a floating tag. For lighter SFT-only flavors a thinner image —

`rocm/pytorch:rocm7.2.1_ubuntu24.04_py3.12_pytorch_release_2.9.1` — is acceptable.

(AMD ROCm)

Above the container, the layered Python stack is `torch==2.9.1+rocm7.2.1.lw` (from `repo.radeon.com/rocm/manylinux/rocm-rel-7.2.1`), *not* `download.pytorch.org/whl/nightly/rocm7.2`, because the AMD-validated wheels are reproducible while nightlies churn), `triton==3.5.1+rocm7.2.1`, `transformers>=4.46,<4.50`, `accelerate>=1.0`, `peft>=0.13`, `trl>=0.12`, `datasets>=3.0`, `optimum>=1.24`, `optimum-amd` from main, `amd-quark>=0.11.1`, `auto-gptq` from `huggingface.github.io/autogptq-index/whl/rocm573`, `bitsandbytes` built from `github.com/ROCm/bitsandbytes` branch `rocm_enabled_multi_backend` with `-DBNB_ROCM_ARCH="gfx942;gfx950" -DCOMPUTE_BACKEND=hip`, `flash-attn` built from `Dao-AILab/flash-attention` upstream with both CK and Triton backends compiled (`FLASH_ATTENTION_TRITON_AMD_ENABLE=TRUE`), and `vllm` ROCm 7.0+ wheels for serving. **Numpy must be pinned** `<2.0` against torch 2.9 ROCm wheels. (AMD ROCm + 8)

The runtime environment is non-negotiable on MI300X: `HSA_NO_SCRATCH_RECLAIM=1`, `NVTE_CK_USES_BWD_V3=1`, `NVTE_CK_IS_V3_ATOMIC_FP32=1`, `PRIMUS_TURBO_ATTN_V3_ATOMIC_FP32=1`, `HIP_FORCE_DEV_KERNARG=1`, `PYTORCH_ROCM_ARCH=gfx942`, `NCCL_MIN_NCHANNELS=112` for sub-8-GPU jobs, `GPU_MAX_HW_QUEUES=1` for multi-GPU stability, and `numa_balancing` off in `/proc/sys/kernel`. **Pin to ROCm 7.2.1 or later** — 7.1.x has a documented bf16/fp16 GEMM cu-fallback regression on gfx942 that costs ~600 ms per call through hipBLASLt heuristic scans. The deprecated `ROCm/Megatron-LM` Docker is being retired in favor of Primus; adopt Primus from day zero to avoid migration debt. (AMD ROCm + 7)

The Qwen3 family targeting decision

Qwen3.6 is real, contrary to the user's implicit doubt. The open-weight cadence reads Qwen3 (April 2025, original 8 dense + 2 MoE checkpoints) → Qwen3-2507 split refresh (July 2025, 256K native context) → Qwen3-Next-80B-A3B (September 2025, hybrid Gated DeltaNet + sparse MoE) → Qwen3-Coder/VL/Omni/Guard/ASR/Image (Aug–Oct 2025) → Qwen3.5 family (Feb 16 2026,

unified text+vision backbone, flagship 397B-A17B) → **Qwen3.6** (April 2026, currently the newest open-weight checkpoint as of May 5 2026). The two open Qwen3.6 checkpoints are `Qwen/Qwen3.6-27B` (dense, released April 22 2026) and `Qwen/Qwen3.6-35B-A3B` (MoE, released April 16 2026), both Apache 2.0, both multimodal `image-text-to-text`, both 262,144 native context extendable to ~1M via YaRN, both default-thinking with no `/think` `/no_think` soft switch (use `chat_template_kwargs={"enable_thinking": False}` instead), and both introduce **Thinking Preservation** (`preserve_thinking=True`) for agentic multi-turn KV-cache reuse. Vocab grew from 151,936 in Qwen3 to 248,320 in Qwen3.6 to support 201 languages.

LM Studio +2 +9

mindXtrain's **default targets** for the hackathon demo should be a tier of three: **Qwen3-8B** as the headline single-MI300X full-FT case (fits one card with bs=8 seq=4096, AdamW, bf16, ~80 GB peak, 12–20k tok/s on Primus-Turbo); **Qwen3-32B** as the FSDP-2 4×MI300X full-FT showpiece (bs=2, seq=4096, ~5–9 hours per 1B tokens at FP8 with TE-CK); and **Qwen3.6-35B-A3B** as the latest-and-greatest MoE LoRA case (experts-only adapters, gate frozen, 2×MI300X with EP enabled). Qwen3-235B-A22B-Thinking-2507 is the dramatic-but-impractical option — LoRA on FP8/MXFP4-quantized weights across 8×MI300X is feasible at ~5–12k tok/s; full FT requires multi-node and is out of hackathon scope. `APXML`

Two Qwen3-specific landmines deserve naming. First, hybrid Gated DeltaNet layers in Qwen3-Next, Qwen3.5 and Qwen3.6 do not use standard SDPA — they require `flash-linear-attention` and `causal-conv1d`, and on ROCm both need community wheels or source builds; without them you fall back to a slow PyTorch reference path. Second, the **MoE router gate** must be frozen during fine-tuning; thawing it almost always diverges. Both pitfalls belong in mindXtrain's autotune as hard rules, not user-facing knobs. `Hugging Face`

The Qwen team's preferred RL algorithm is **GRPO** (Qwen3 launch/2507) and its successor **GSPO** (Qwen3-Next/3.5/3.6, recommended for hybrid + sparse MoE stability). AMD has published an end-to-end ROCm + TRL + vLLM + DeepSpeed GRPO recipe on MI300X using GSM8K and Qwen2.5-1.5B-Instruct — that recipe transposes one-to-one onto Qwen3-8B and is the safest single-node demo path. `ROCm Blogs`

What the open-source landscape gives us, and what it doesn't

The framework survey produced a clear ranking. **Axolotl** (Apache-2.0, YAML-driven, 70k+ models, March 2026 added Qwen3.5/Qwen3.5-MoE, ND parallelism, FP8 via torchao) is the right *YAML skeleton* for mindXtrain's multi-GPU SFT/DPO/ORPO/GRPO/QAT path; ROCm support is second-class but the AI-DarwinLabs `amd-support` branch ships a working `requirements-amd.txt` and AMD itself published a Dockerfile.rocm walkthrough. **LLaMA-Factory** (Apache-2.0, 70.6k stars, Qwen team's own preferred trainer, AMD's own MI300X tutorial uses it) is the right *Qwen3-specific recipe library* — supports every Qwen3 variant including 2507, VL, Omni and Next, plus PPO/DPO/KTO/ORPO/SimPO/GRPO. **torch tune** (BSD-3, AMD-CI-tested, the cleanest codebase) is the right *modular reference implementation*, weak only because Qwen3

recipes are not yet upstream — a ~200-line PR that mindXtrain should ship as a side-deliverable. **Unsloth** (Apache-2.0, official AMD partnership via OneClickAMD, fastest single-GPU experience) belongs as an **opt-in backend** for single-MI300X jobs where its 16-bit LoRA path beats everyone else, with the caveat that bitsandbytes 4-bit is currently unstable on ROCm 7.x and Unsloth correctly disables it. Below those four sit **TRL + PEFT + Accelerate** as mandatory dependencies (every higher-level trainer wraps them; Accelerate is first-class on ROCm without code changes), **Optimum-AMD** as the mandatory Flash-Attention-2/GPTQ/`amdrun`-topology shim (now MIT-licensed, Apache-compatible), **DeepSpeed** as the first-class ZeRO/MoE backend on ROCm 6+, **vLLM-ROCm** and **SGLang** as first-class rollout engines for GRPO and serving (vLLM ROCm CI went live December 2025, SGLang ships explicit `rocm/sgl-dev:v0.5.8.post1-rocm720-mi30x` images), and **AMD-AGI/Primus + Primus-Turbo** as the right pretraining-scale fallback when mindXtrain is asked to do continued pretraining at >70 B parameters. NeMo is disqualified (CUDA-only TransformerEngine dependency), the AMD GPT-NeoX branch is three years stale, Megatron-DeepSpeed is superseded, and Lamini is CC-BY-NC and disqualified by license. `Axolotl + 14`

The whitespace mindXtrain fills is precisely the seven gaps no existing framework owns: **(1)** MI300X-aware automated hyper-parameter selection driven by a 60-second micro-benchmark; **(2)** a one-line install matrix that pins a known-working ROCm/PyTorch/Flash-Attn/bitsandbytes/Unsloth tuple per ROCm version; **(3)** Qwen3-family torchtune recipes (upstreamable PR); **(4)** a unified `mindxtrain.yaml` schema that compiles down to either Axolotl or Unsloth backends based on cost/scale; **(5)** a native MI300X 4-bit path that bypasses bitsandbytes via Quark FP8/MXFP4 + LoRA-on-quantized weights; **(6)** auto-orchestration of vLLM-ROCm/SGLang rollout engines on idle GPUs of the same MI300X node for in-the-loop GRPO; and **(7)** a hackathon-grade reproducibility manifest — a "training receipt" capturing ROCm version, gfx target, every git SHA, the Docker digest, the exact YAML, and dataset hashes.

`AMD ROCm`

mindXtrain architecture

mindXtrain is structured as five concentric layers. The **CLI layer** (`mindxtrain/cli.py`) exposes `mindxtrain init`, `mindxtrain bench` (the autotune micro-benchmark), `mindxtrain dataset prep`, `mindxtrain train`, `mindxtrain eval`, `mindxtrain quantize`, `mindxtrain serve`, `mindxtrain publish` and `mindxtrain receipt`. Each command consumes a single Pydantic-validated YAML config that supersedes Axolotl's, LLaMA-Factory's and torchtune's schemas; mindXtrain's job is to compile that config down to whichever backend YAML the underlying trainer expects. The **autotune layer** runs first — it inspects the (model, dataset, seq-len, GPU topology) tuple, executes a short profiled forward+backward to measure attention kernel throughput, GEMM heuristics, AllReduce bandwidth and HBM usage, then writes a `mindxtrain.tuned.yaml` with backend-specific overrides (CK vs Triton attention, AITER MoE on/off, NCCL_MIN_NCHANNELS, FSDP shard, LoRA rank ceiling, gradient-checkpointing policy). This layer is **AOT-only**: the autotune produces a static plan that is loaded at training start;

JIT autotune is forbidden in production to keep training reproducible, in line with the user's existing mindX AutoTune discipline. The **dataset layer** wraps `(datasets)`, deduplicates with MinHash + SemDeDup, applies quality filtering, packs sequences (pack-to-cutoff Qwen3-style), shards for FSDP, and emits content-addressed Lighthouse Storage CIDs for provenance. The **training layer** dispatches to one of four backends — `(axolotl)` (multi-GPU SFT/DPO/ORPO/GRPO), `(unsloth)` (single-MI300X fast LoRA/GRPO), `(torchtune)` (modular recipes), or `(primus)` (pretraining-scale Megatron/TorchTitan) — selected by a `(backend:)` key with a sensible default chosen by autotune. Accelerate is the launcher in all cases except primus. The **artifact + integration layer** quantizes via Quark (FP8 / MXFP4 / INT4-FP8 two-level), evaluates with `(lm-evaluation-harness)` plus user-supplied custom evals with automatic regression detection against a baseline checkpoint, generates a model card, pushes to the HF org, mirrors the safetensors to Lighthouse Storage with a CID receipt, registers the trained model with the **mindX cognitive API** `(mindx.pythai.net)` as a new agent capability, lists it on **AgenticPlace** `(agenticplace.pythai.net)`, allocates an **ENS subname** under `(bankon.eth)` for the resulting agent, and wires **x402 Algorand micropayments via parsec/parsec-wallet** for the training-as-a-service billing flow. Optional decentralized compute hooks for Bacalhau, Akash, and io.net are stubbed in `(mindxtrain/compute/)` so a job YAML can opt into running on a non-AMD-Cloud provider. `(AMD ROCm)`

The **CLI surface** is deliberately small. `(mindxtrain init <project>)` scaffolds a flat `snake_case` directory and a starter YAML. `(mindxtrain bench --model Qwen/Qwen3-8B --hardware mi300x --gpus 1)` runs the autotune. `(mindxtrain train -c config.yaml)` runs the full pipeline. `(mindxtrain serve --checkpoint ./out/last --backend vllm-rocm)` spins a vLLM-ROCM endpoint with the right tool-call and reasoning parsers (`(hermes)` + `(qwen3)`/`(deepseek_r1)` for non-coder Qwen3 models, `(qwen3_coder)` for Coder variants). `(mindxtrain publish)` pushes everything — checkpoint, model card, eval report, training receipt, Lighthouse CID — to HF, mindX, AgenticPlace and the BANKON ENS layer in one call. `(mindxtrain receipt <run-id>)` re-emits the manifest for any past run. `(vLLM)` `(AMD)`

The **config schema** is a single YAML with five top-level sections — `(meta)`, `(model)`, `(data)`, `(train)`, `(serve)` — each strongly typed by Pydantic. A canonical Qwen3-8B SFT-with-LoRA-on-1×MI300X config is reproduced below in the snippets section. Cross-cutting fields like `(seed)`, `(provenance.lighthouse_endpoint)`, and `(hardware.gfx_arch)` apply globally. Each training method (full FT, LoRA, QLoRA, DPO, ORPO, GRPO, GSPO, KTO, continued pretraining, multimodal Qwen3-VL/Omni/Image) is a discriminated union under `(train.method)`. **Hyperparameter search** uses Optuna by default (TPE sampler, MedianPruner) with the search space bounded by autotune-derived hard limits; Ray Tune is supported when a Kubernetes cluster is available; W&B Sweeps is the third option for teams already on W&B. The bounding rule is non-negotiable: autotune's profiled max batch size is the upper bound on `(per_device_train_batch_size)`, full stop.

Hackathon-winning submission strategy

The 90-second demo storyline is concrete. Open with terminal: a single `mindxtrain train -c demo_qwen3_8b_sft.yaml` command. Cut to the autotune dashboard streaming MI300X micro-benchmark output (CK FA forward TFLOPS, hipBLASLt heuristic times, RCCL bus bandwidth) for 60 seconds. Cut to the training loop streaming loss + tok/s + MFU (target >40% MFU on Qwen3-8B BF16 with Primus-Turbo, comparable to AMD's published Llama-3.1-8B numbers). Cut to a side-by-side cost slide: **MI300X at \$1.99/hr × 1 GPU × 3 hours = ~\$6 versus H100 at \$4/hr × 2 GPUs × 4 hours = ~\$32**, with the headline "4× cheaper, can't OOM at 192 GB." Cut to the resulting model live in the HF Space chatting in thinking mode, then show `mindxtrain receipt` printing the full provenance manifest. Closing slide is one diagram of mindXtrain → mindX → AgenticPlace → BANKON. The story takes 90 seconds, leaves 60-90 seconds for Q&A in the 3-minute video budget. (LabLab) (LabLab)

The deliverables checklist mapped to lablab's submission rubric is: a working prototype deployed as a Hugging Face Space inside the `lablab-ai-amd-developer-hackathon` HF org (Space name: `mindxtrain-demo`); a 2-3 minute demo video posted to YouTube and tweeted from the codephreak account tagging @lablab @AIatAMD @AMDROCM @huggingface @Alibaba_Qwen (auto-qualifies for Build-in-Public); a pitch deck of 5-7 slides Sequoia-style; the GitHub repo at `pythai/mindxtrain` MIT-licensed (or Apache-2.0 — verify the lablab page's "MIT-only" requirement on the live form, with Apache-2.0 as the codephreak-preferred fallback that may need a license-compat note); the lablab platform submission form filled with a 50-char title `mindXtrain - one-command Qwen3 on MI300X`, a 255-char short description, a 100-word-minimum long description naming every AMD library used, the cover image, the video URL, the GitHub URL, and the demo Space URL; verification of AMD AI Developer Program membership for the \$100 cloud credits; and at least two technical X posts during the build window plus a written ROCm feedback note for Build-in-Public eligibility. (LabLab + 12)

The README structure for the GitHub repo is `# mindXtrain` headline, one-line tagline, a 30-second quick-start (`pipx install mindxtrain && mindxtrain init demo && mindxtrain train`), an architecture diagram (Mermaid), a benchmark table comparing mindXtrain to Axolotl/LLaMA-Factory/Unsloth/torchTune on Qwen3-8B-on-1×MI300X for the metrics tok/s, time-to-eval-loss-1.5, MFU, and total \$; an "AMD stack exploited" section enumerating ROCm 7.2.1, AOTriton, AITER, Composable Kernel, hipBLASLt with offline tuning, Flash-Attention-CK, RCCL, Optimum-AMD, Quark, Primus-Turbo, vLLM-ROCM, and SGLang with one-line citations; a "what makes this MI300X-native" section pointing at the autotune; a license header; a roadmap; and an acknowledgments block thanking AMD, Hugging Face, and the Qwen team. The benchmark numbers to chase on the hero workload (Qwen3-8B SFT, 1×MI300X, bs=8, seq=4096, BF16, AdamW, 1B tokens) are **>15k tok/s, MFU >40%, time-to-loss-1.5 <90 minutes, total cost <\$3**. Hit those and the cost slide writes itself.

The comparison table mindXtrain prints in its README is the differentiator artifact: rows are Axolotl, LLaMA-Factory, Unsloth, torchTune, Primus, mindXtrain; columns are *one-command install on ROCm 7.2.1, MI300X auto-tune, Qwen3.6 day-zero recipe, FP8 via Quark, x402*

micropayments, decentralized fallback, training receipt manifest. mindXtrain is the only row with all seven cells filled.

The risks and mitigations are well-defined. **Risk:** bitsandbytes 4-bit instability on ROCm 7.x. **Mitigation:** ship the Quark FP8 + LoRA path as default; bitsandbytes only as opt-in. **Risk:** Qwen3.6 GDN layers need flash-linear-attention/causal-conv1d which lack official ROCm wheels. **Mitigation:** build wheels in CI, ship in the Podman image, pin commit SHAs; fall back to PyTorch reference if build fails. **Risk:** lablab requires MIT license per submission rules but codephreak prefers Apache 2.0. **Mitigation:** dual-license the submission repo MIT for hackathon compliance, with a NOTICE file pointing at the upstream Apache-2.0 mindX ecosystem; reconcile post-hackathon. **Risk:** 60-second autotune window may not cover MoE expert imbalance. **Mitigation:** emit a warning in the receipt and run a longer autotune for Qwen3-30B-A3B / Qwen3.6-35B-A3B / Qwen3-235B-A22B specifically. **Risk:** vLLM-ROCm Triton autotune can stall first-batch on cold start. **Mitigation:** warm-up batch in `mindxtrain serve` before exposing the endpoint; or fall back via `VLLM_USE_TRITON_FLASH_ATTN=0`. `LabLab + 2`

Repository skeleton

The cypherpunk2048 standard is flat snake_case throughout, Apache-2.0 license header on every file, no proprietary lock-in, no upgradeable proxies in any Solidity, no EOA admin keys, Foundry for Solidity build, Podman over Docker:

```

mindxtrain/
├─ pyproject.toml
├─ README.md
├─ LICENSE # Apache-2.0 (hackathon submission may dual-
license MIT)
├─ NOTICE
├─ mindxtrain/
│   ├─ __init__.py
│   ├─ cli.py # entry point (typer-based)
│   ├─ config.py # Pydantic schema
│   └─ autotune/
│       ├─ __init__.py
│       ├─ benchmark.py # 60-second MI300X probe
│       ├─ attention_probe.py # CK vs Triton vs AITER selection
│       ├─ gemm_probe.py # hipBLASLt offline tune trigger
│       ├─ rccl_probe.py # bus-bandwidth measurement
│       └─ plan.py # writes mindxtrain.tuned.yaml (AOT)
├─ dataset/
│   ├─ load.py
│   ├─ dedupe_minhash.py
│   ├─ dedupe_semdedup.py
│   ├─ quality_filter.py
│   ├─ tokenize.py
│   ├─ pack.py
│   └─ shard.py
├─ train/
│   ├─ dispatch.py # picks backend
│   ├─ backend_axolotl.py
│   ├─ backend_unsloth.py
│   ├─ backend_torch tune.py
│   ├─ backend_primus.py
│   └─ recipes/
│       ├─ qwen3_8b_sft_lora.yaml
│       ├─ qwen3_8b_sft_full.yaml
│       ├─ qwen3_32b_full_fsdp.yaml
│       ├─ qwen3_32b_dpo.yaml
│       ├─ qwen3_32b_orpo.yaml
│       ├─ qwen3_32b_grpo.yaml
│       ├─ qwen3_30b_a3b_lora.yaml
│       ├─ qwen3_6_27b_lora.yaml
│       ├─ qwen3_6_35b_a3b_lora.yaml
│       ├─ qwen3_v1_8b_sft.yaml
│       └─ qwen3_8b_cpt.yaml

```



```

├─ eval/
│   ├── harness.py          # lm-evaluation-harness wrapper
│   ├── custom_eval.py
│   └─ regression.py
├─ quantize/
│   ├── quark_fp8.py
│   ├── quark_mxfp4.py
│   └─ gptq_rocm.py
├─ serve/
│   ├── vllm_rocm.py
│   ├── sglang_rocm.py
│   └─ parsers.py          # hermes / qwen3_coder / qwen3 reasoning
├─ publish/
│   ├── hf_hub.py
│   ├── lighthouse.py      # IPFS / Filecoin via Lighthouse Storage
│   ├── mindx_register.py  # POST to mindx.pythai.net
│   ├── agenticplace_list.py # POST to agenticplace.pythai.net
│   └─ bankon_ens.py       # ENS subname under bankon.eth
├─ billing/
│   ├── x402_algorand.py   # parsec/parsec-wallet integration
│   └─ pricing.py
├─ compute/
│   ├── amd_dev_cloud.py
│   ├── tensorwave.py
│   ├── bacalhau.py        # optional decentralized
│   ├── akash.py
│   └─ ionet.py
├─ telemetry/
│   ├── prometheus_exporter.py
│   ├── otel_hooks.py
│   └─ energy.py           # MI300X power tracking via rocm-smi
├─ receipt/
│   ├── manifest.py        # full provenance record
│   └─ verify.py
├─ chain_map/
│   └─ allchain.py         # consumes
agenticplace.pythai.net/allchain.html
├─ infra/
│   ├── podman/
│   │   ├── containerfile_train    # FROM rocm/primus:v26.2
│   │   ├── containerfile_serve    # FROM rocm/vllm-dev:rocm7.2.1
│   │   └─ digest.lock
│   └─ compose/
│       └─ compose_dev.yaml

```

```

├── k8s/
│   └── train_job.yaml
├── contracts/
│   ├── foundry.toml
│   ├── src/
│   │   ├── mindxtrain_registry.sol # immutable, no proxy, no admin
│   │   └── x402_receiver.sol
│   ├── script/
│   └── test/
├── examples/
│   ├── demo_qwen3_8b_sft.yaml
│   ├── demo_qwen3_6_27b_lora.yaml
│   └── demo_grpo_gsm8k.yaml
├── tests/
│   ├── test_autotune.py
│   ├── test_config.py
│   ├── test_recipes.py
│   └── test_receipt.py
├── docs/
│   ├── architecture.md
│   ├── benchmarks.md
│   └── hackathon_submission.md
├── .github/workflows/
│   ├── ci_lint.yml
│   ├── ci_rocm_smoke.yml # runs on self-hosted MI300X runner
│   └── publish_pypi.yml

```

Critical code snippets

pyproject.toml (excerpt):

```
toml
```

```
[project]
name = "mindxtrain"
version = "0.1.0"
description = "One-command Qwen3 fine-tuning on AMD MI300X"
requires-python = ">=3.12"
license = { text = "Apache-2.0" }
authors = [{ name = "Gregory (codephreak)", email = "codephreak@pythai.net" }]
dependencies = [
    "typer>=0.12",
    "pydantic>=2.7",
    "pyyaml>=6.0",
    "transformers>=4.46,<4.50",
    "accelerate>=1.0",
    "peft>=0.13",
    "trl>=0.12",
    "datasets>=3.0",
    "optimum>=1.24",
    "optimum-amd",
    "amd-quark>=0.11.1",
    "lm-eval>=0.4.5",
    "optuna>=3.6",
    "prometheus-client>=0.20",
    "opentelemetry-api>=1.27",
    "lighthouse-web3>=0.1",
    "py-algorand-sdk>=2.6",
    "ens>=0.5",
    "datasketch>=1.6",          # MinHash dedupe
    "numpy<2.0",
]
```

```
[project.optional-dependencies]
axolotl = ["axolotl @ git+https://github.com/axolotl-ai-cloud/axolotl@main"]
unsloth = ["unsloth"]
torchtune = ["torchtune"]
primus = ["primus @ git+https://github.com/AMD-AGI/Primus@v26.2"]
```

```
[project.scripts]
mindxtrain = "mindxtrain.cli:app"
```

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

```
[tool.ruff]
line-length = 100
target-version = "py312"
```

mindxtrain/cli.py (entry-point sketch):

python

```

# SPDX-License-Identifier: Apache-2.0
"""mindXtrain CLI — one-command Qwen3 fine-tuning on AMD MI300X."""
from __future__ import annotations
import typer
from pathlib import Path
from mindxtrain.config import load_config
from mindxtrain.autotune.benchmark import run_benchmark
from mindxtrain.autotune.plan import write_tuned_plan
from mindxtrain.train.dispatch import dispatch_training
from mindxtrain.eval.harness import run_eval
from mindxtrain.quantize.quark_fp8 import quantize_fp8
from mindxtrain.serve.vllm_rocm import serve_vllm
from mindxtrain.publish.hf_hub import publish_to_hf
from mindxtrain.publish.lighthouse import publish_to_lighthouse
from mindxtrain.publish.mindx_register import register_with_mindx
from mindxtrain.publish.agenticplace_list import list_on_agenticplace
from mindxtrain.publish.bankon_ens import allocate_ens_subname
from mindxtrain.receipt.manifest import emit_receipt

app = typer.Typer(no_args_is_help=True, add_completion=False)

@app.command()
def init(project: str) -> None:
    """Scaffold a new mindXtrain project (flat snake_case)."""
    Path(project).mkdir(parents=True, exist_ok=False)
    Path(f"{project}/config.yaml").write_text(_starter_yaml(project))
    typer.echo(f"initialized {project}/")

@app.command()
def bench(
    model: str = typer.Option(..., "--model"),
    hardware: str = typer.Option("mi300x", "--hardware"),
    gpus: int = typer.Option(1, "--gpus"),
    seq_len: int = typer.Option(4096, "--seq-len"),
    out: Path = typer.Option(Path("mindxtrain.tuned.yaml"), "--out"),
) -> None:
    """Run the 60-second MI300X autotune probe; emits an AOT plan."""
    measurements = run_benchmark(model=model, hardware=hardware, gpus=gpus, seq_len=seq_len)
    write_tuned_plan(measurements, out)
    typer.echo(f"wrote tuned plan to {out}")

@app.command()
def train(config: Path = typer.Option(..., "-c", "--config")) -> None:

```

```

"""Run the full pipeline (autotune → train → eval → quantize → publish)."""
cfg = load_config(config)
if cfg.autotune.enabled and not cfg.autotune.plan_path.exists():
    bench(model=cfg.model.name, hardware=cfg.hardware.name,
          gpus=cfg.hardware.gpus, seq_len=cfg.data.seq_len,
          out=cfg.autotune.plan_path)
run_id = dispatch_training(cfg)
eval_report = run_eval(cfg, run_id)
if cfg.quantize.enabled:
    quantize_fp8(cfg, run_id)
if cfg.publish.enabled:
    hf_url = publish_to_hf(cfg, run_id, eval_report)
    cid = publish_to_lighthouse(cfg, run_id)
    register_with_mindx(cfg, run_id, hf_url, cid)
    list_on_agenticplace(cfg, run_id, hf_url)
    allocate_ens_subname(cfg, run_id)
    emit_receipt(cfg, run_id, eval_report)

@app.command()
def serve(
    checkpoint: Path = typer.Option(..., "--checkpoint"),
    backend: str = typer.Option("vllm-rocm", "--backend"),
    port: int = typer.Option(8000, "--port"),
) -> None:
    """Serve a trained checkpoint on vLLM-ROCM or SGLang with correct parsers."""
    if backend == "vllm-rocm":
        serve_vllm(checkpoint, port=port)
    else:
        from mindxtrain.serve.sglang_rocm import serve_sglang
        serve_sglang(checkpoint, port=port)

if __name__ == "__main__":
    app()

```

examples/demo_qwen3_8b_sft.yaml1 (the hackathon hero config, fits one MI300X):

yaml

```
# SPDX-License-Identifier: Apache-2.0
meta:
  project: mindxtrain_demo
  run_name: qwen3_8b_sft_demo
  seed: 2048
  license: apache-2.0

hardware:
  name: mi300x
  gfx_arch: gfx942
  gpus: 1
  expected_hbm_gb: 192

autotune:
  enabled: true
  plan_path: ./out/mindxtrain.tuned.yaml
  budget_seconds: 60
  policy: aot_only          # JIT autotune forbidden in production

model:
  name: Qwen/Qwen3-8B
  attn_implementation: flash_attention_2  # CK backend by default; autotune may over
  torch_dtype: bfloat16
  trust_remote_code: false

data:
  source: hf
  hf_id: HuggingFaceH4/ultrachat_200k
  split: train_sft
  seq_len: 4096
  packing: true
  dedupe:
    minhash: { threshold: 0.85 }
    semdedup: { threshold: 0.95, model: sentence-transformers/all-MiniLM-L6-v2 }
  shard:
    num_shards: 1

train:
  backend: axolotl          # autotune may flip to unsloth for single-GPU LoRA
  method:
    kind: lora
    r: 16
    alpha: 32
```

```
dropout: 0.0
target_modules: [q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj]
optimizer:
  name: adamw_torch_fused
  lr: 1.0e-4
  betas: [0.9, 0.95]
  weight_decay: 0.1
  grad_clip: 1.0
schedule:
  type: cosine
  warmup_ratio: 0.03
  epochs: 3
batch:
  per_device: 8
  grad_accum: 4
precision: bf16
gradient_checkpointing: true
flash_attention:
  backend: ck # ck | triton | aiter - autotune picks
fsdp: { enabled: false } # single GPU
env:
  HSA_NO_SCRATCH_RECLAIM: "1"
  NVTE_CK_USES_BWD_V3: "1"
  NVTE_CK_IS_V3_ATOMIC_FP32: "1"
  PRIMUS_TURBO_ATTN_V3_ATOMIC_FP32: "1"
  NCCL_MIN_NCHANNELS: "112"
  HIP_FORCE_DEV_KERNARG: "1"
  PYTORCH_ROCM_ARCH: "gfx942"

eval:
  harness:
    tasks: [mm1u, gsm8k, ifeval, humaneval]
    fewshot: 5
  regression:
    baseline: Qwen/Qwen3-8B
    threshold_pct: -1.0 # fail if any task drops more than 1 pct

quantize:
  enabled: true
  scheme: quark_fp8 # quark_fp8 | quark_mxfp4 | gptq_rocm
  ptpc: true # PTPC FP8 GEMM (15-30% faster than BlockScale on MI300X)

serve:
  backend: vllm-rocm
```



```
reasoning_parser: deepseek_r1    # qwen3 for 3.5/3.6
tool_call_parser: hermes         # qwen3_coder for Coder family
tensor_parallel: 1

publish:
  enabled: true
  hf:
    repo: pythai/qwen3-8b-mindxtrain-demo
    private: false
  lighthouse:
    api_key_env: LIGHTHOUSE_API_KEY
  mindx:
    api_url: https://mindx.pythai.net/v1/agents
    register_as_capability: true
  agenticplace:
    api_url: https://agenticplace.pythai.net/v1/listings
    chain_map_url: https://agenticplace.pythai.net/allchain.html
  bankon:
    ens_parent: bankon.eth
    subname: qwen3-8b-mindxtrain-demo
  billing:
    x402:
      network: algorand
      asset: USDC
      receiver_via: parsec_wallet
      price_per_1k_tokens: 0.0002

receipt:
  output: ./out/receipt.json
  include:
    [rocm_version, gfx_arch, container_digest, all_git_shas,
     yaml_hash, dataset_cids, eval_report, energy_kwh]
```

Sample Solidity registry stub (Foundry, immutable, no proxy, no EOA admin):

```
solidity
```

```
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.26;

contract MindXTrainRegistry {
    struct Receipt {
        bytes32 yamlHash;
        bytes32 datasetCidHash;
        bytes32 checkpointCidHash;
        bytes32 evalReportHash;
        address publisher;
        uint64 timestamp;
    }

    mapping(bytes32 => Receipt) private _receipts;
    event ReceiptAnchored(bytes32 indexed runId, address indexed publisher, bytes32

function anchor(
    bytes32 runId,
    bytes32 yamlHash,
    bytes32 datasetCidHash,
    bytes32 checkpointCidHash,
    bytes32 evalReportHash
) external {
    require(_receipts[runId].timestamp == 0, "exists");
    _receipts[runId] = Receipt({
        yamlHash: yamlHash,
        datasetCidHash: datasetCidHash,
        checkpointCidHash: checkpointCidHash,
        evalReportHash: evalReportHash,
        publisher: msg.sender,
        timestamp: uint64(block.timestamp)
    });
    emit ReceiptAnchored(runId, msg.sender, yamlHash);
}

function get(bytes32 runId) external view returns (Receipt memory) {
    return _receipts[runId];
}
}
```

No constructor admin, no `Ownable`, no upgradeable proxy, no pause function, no setter — write-once anchoring, EOA-key-free administration. DAIO blockchain deployment is the remaining

piece per the user's standard preferences; mainnet Foundry deploy targets the canonical chain ID resolved through `agenticplace.pythai.net/allchain.html`.

Reconciliation with prior `mindXtrain.md` and `mindXtrain2.md`

Because the two prior design files are not in research context, this document is written as a forward-compatible **continuation**, not a replacement. Three places need explicit reconciliation when the prior files are read alongside this one. **First**, the AMD-track framing pulls the recommended default model away from any GPU-agnostic choice toward Qwen3.6-27B / Qwen3.6-35B-A3B / Qwen3-8B specifically, because Qwen integration is a stackable hackathon prize and Qwen3 has confirmed Day-0 ROCm support; if `mindXtrain.md/mindXtrain2.md` committed to a different default base model, that decision should be revisited for the hackathon submission only and reverted afterwards if needed. **Second**, the AOT-only autotune discipline carried forward from the user's mindX AutoTune work conflicts with any framework default that turns on JIT autotune (Triton autotune in vLLM cold-start, `torch.compile(mode='max-autotune')`) Inductor JIT, MIOpen find-mode); the `mindXtrain` config schema must explicitly disable JIT autotune in production runs and force AOT compilation paths (AOTriton, hipBLASLt offline tune cache, MIOpen `.kdb` pre-warming). **Third**, the BANKON ENS allocation and x402-Algorand billing flow is named here as a first-class integration; if the prior files described `mindXtrain` as standalone, this needs to be elevated from optional to mandatory in the publish step, and the cypherpunk2048 immutability rule (no upgradeable proxies, no EOA admin) must propagate into the on-chain registry contract above. `ROCm Blogs` `AMD ROCm`

Hackathon timeline (today is May 5 2026)

The build window has roughly five days of active engineering left. **May 5 (today)**: register on `lablab.ai`, register the AMD AI Developer Program for the \$100 cloud credits, provision an MI300X via TensorWave bare-metal or AMD Developer Cloud, snapshot the `rocm/primus:v26.2` digest, scaffold the `mindxtrain/` repo with the directory tree above, ship the Pydantic config schema and the Typer CLI, post a Build-in-Public X teaser tagging `@lablab` `@AIatAMD` with a screenshot of `mindxtrain init`. **May 6**: implement the autotune layer — attention probe, GEMM probe, RCCL probe, plan emitter — and run the first end-to-end Qwen3-8B SFT-LoRA on a single MI300X with the demo YAML; capture tok/s and MFU baseline numbers. **May 7**: implement the dataset pipeline (MinHash, SemDeDup, packing, sharding), the eval harness wrapper, and the Quark FP8 PTPC quantization path; ship a second X post showing the autotune-driven config diff with a benchmark vs untuned baseline. **May 8**: implement the publish layer (HF, Lighthouse, mindX register, AgenticPlace listing, BANKON ENS subname), the x402 billing stub, the receipt manifest, and deploy the public HF Space inside `lablab-ai-amd-developer-hackathon`; record the demo video and write the final pitch deck. **May 9**: travel to SF or stream to the on-site session, finalize the `lablab` submission form by 14:00 local Saturday, drive social engagement on the HF Space for the most-likes prize. **May 10**: live demo on stage,

awards, post-mortem. Submit the written ROCm developer-experience feedback note immediately after submission to lock in Build-in-Public eligibility.  

Closing synthesis

mindXtrain wins this hackathon by being the only entry that operationalizes the *entire* AMD training stack — ROCm 7.2.1, AOTriton, AITER, Composable Kernel, hipBLASLt, RCCL, Optimum-AMD, Quark, Primus-Turbo, vLLM-ROCM, SGLang — behind a single CLI, with a defensible **AOT autotune** that nobody else in the ecosystem ships, against the **latest open Qwen3.6 checkpoints** (which the user correctly remembered as real and which most public summaries lag), with cost numbers that make MI300X look obviously cheaper than H100 for the workloads in scope. The cypherpunk2048 discipline — Apache 2.0, flat snake_case, Podman, immutable contracts, no proxies, no EOA admin — is preserved end-to-end, and the integration plumbing into mindX, AgenticPlace, BANKON-ENS and x402-Algorand is wired without locking in a proprietary dependency. The remaining engineering is entirely tractable in the five-day window, and every external dependency named in this blueprint has either a verified ROCm-first-class status or a documented community workaround. Ship it.

Citations

Hackathon: lablab.ai/ai-hackathons/amd-developer · amd.com/en/developer/resources/technical-articles/2026/build-across-the-ai-stack--join-the-amd-x-lablab-ai-hackathon-.html · luma.com/afz0aeq8 · huggingface.co/lablab-ai-amd-developer-hackathon · lablab.ai/ai-articles/from-zero-to-ai-builder-amd-developer-program · lablab.ai/ai-tutorials/amd-developer-cloud-host-llm-vllm

Frameworks: github.com/axolotl-ai-cloud/axolotl · github.com/AI-DarwinLabs/axolotl · github.com/hiyouga/LLaMA-Factory · github.com/unslothai/unsloth · github.com/pytorch/torch-tune · github.com/huggingface/trl · github.com/huggingface/peft · github.com/huggingface/accelerate · github.com/huggingface/optimum-amd · github.com/microsoft/DeepSpeed · github.com/AMD-AGI/Primus · github.com/AMD-AGI/Primus-Turbo · github.com/vllm-project/vllm · github.com/sgl-project/sglang

AMD stack: rocm.docs.amd.com/en/latest/about/release-notes.html · rocm.docs.amd.com/en/latest/compatibility/compatibility-matrix.html · rocm.docs.amd.com/projects/install-on-linux/en/latest/install/3rd-party/pytorch-install.html · github.com/ROCm/bitsandbytes (rocm_enabled_multi_backend) · github.com/Dao-AI-Lab/flash-attention · github.com/ROCm/aotriton · github.com/ROCm/aiter · github.com/ROCm/composable_kernel · github.com/amd/Quark · quark.docs.amd.com · rocm.blogs.amd.com/software-tools-optimization/mi300x-rccl-xgmi · rocm.blogs.amd.com/software-tools-optimization/vllm-omni · rocm.blogs.amd.com/software-tools-optimization/llm-grpo-rocm · rocm.blogs.amd.com/artificial-intelligence/qwen3-day0-amd

· rocm.blogs.amd.com/artificial-intelligence/torch-tune ·
www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html ·
www.amd.com/en/products/accelerators/instinct/mi350.html ·
www.amd.com/en/developer/resources/technical-articles/2026/day-0-support-for-qwen-3-5-on-amd-instinct-gpus.html · huggingface.co/blog/huggingface-and-optimum-amd ·
huggingface.co/blog/microsoft-collaboration · huggingface.co/amd ·
huggingface.co/docs/optimum/en/amd/index · arxiv.org/pdf/2510.27583 ·
newsletter.semianalysis.com/p/mi300x-vs-h100-vs-h200-benchmark-part-1-training

Qwen3: arxiv.org/abs/2505.09388 · arxiv.org/abs/2509.17765 · arxiv.org/abs/2511.21631 ·
qwenlm.github.io/blog/qwen3 · qwen.ai/blog?id=qwen3-next · qwen.ai/blog?id=qwen3.5 ·
qwen.ai/blog?id=qwen3.6-27b · qwen.ai/blog?id=qwen3.6-35b-a3b ·
github.com/QwenLM/Qwen3 · github.com/QwenLM/Qwen3-Coder ·
github.com/QwenLM/Qwen3-VL · github.com/QwenLM/Qwen3-Omni ·
github.com/QwenLM/Qwen3.6 · huggingface.co/Qwen · huggingface.co/Qwen/Qwen3-8B ·
huggingface.co/Qwen/Qwen3-32B · huggingface.co/Qwen/Qwen3-235B-A22B-Thinking-2507 ·
huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct · huggingface.co/Qwen/Qwen3-Coder-480B-A35B-Instruct ·
huggingface.co/Qwen/Qwen3.6-27B · huggingface.co/Qwen/Qwen3.6-35B-A3B · qwen.readthedocs.io/en/latest/getting_started/quickstart.html ·
qwen.readthedocs.io/en/latest/getting_started/concepts.html · www.lmsys.org/blog/2026-02-11-Qwen-latency · docs.unsloth.ai/models/qwen3-how-to-run-and-fine-tune ·
unsloth.ai/docs/models/qwen3.6